Práctica 1 Optativa

Webpack



Alberto Torres Castellano

2 de octubre del 2026

Introducción

Introducción 1
Qué es Webpack y para qué sirve 2
Conceptos clave 3
1. Entry (punto de entrada) 3
2. Output (salida) 3
3. Loaders 3
4. Plugins 4
5. Mode (development / production) 4
6. Code splitting y Hot Module Replacement (HMR) 4
Requisitos e instalación 5
Instalación 5
Ejemplo básico de uso 8
Ventajas e inconvenientes 13
Recursos y bibliografía 14

Qué es Webpack y para qué sirve

Esta tecnología nos permite coger todos los archivos de un proyecto front-end (JavaScript, HTML, CSS, imágenes, fuentes o plantillas) y lo convierte en un número reducido de archivos finales optimizados para que el navegador los cargue de forma eficiente, es decir Webpack se encarga de empaquetar esos archivos.

El problema que resuelve Webpack es permitir usar módulos (import/export o require de CommonJS) en código que se puede ejecutar en el navegador, resolviendo así las dependencias entre archivos. Otra cosa que permite no es solo empaquetar JavaScript, sino cualquier recurso, ya sea CSS, imágenes, etc., a través de un sistema de loaders y plugins. permitiendo optimizar el resultado final.

Webpack es un proyecto de código abierto mantenido por la organización Webpack en GitHub creado originalmente por Tobias Koppers y sostenido económicamente en parte a través de OpenCollective. La versión actual es la 5.x con publicaciones frecuentes de parches, este se instala siempre indicando esta rama en “package.json”.

```
"devDependencies": {
        "webpack": "^5.107.0",
        "webpack-cli": "^5.1.4"
}
```

Conceptos clave

Antes de tocar Webpack conviene tener claras estas seis ideas:

1. Entry (punto de entrada)

Es el archivo por el que Webpack "empieza a tirar del hilo": desde ahí sigue todos los `import`/`require` para saber qué otros archivos necesita incluir.

```
js

// webpack.config.js

entry: './src/index.js'
```

2. Output (salida)

Dónde y con qué nombre se genera el archivo final ya empaquetado.

```
js

output: {

  filename: 'bundle.js',

  path: path.resolve(__dirname, 'dist')

}
```

3. Loaders

Webpack, por sí solo, solo entiende JavaScript y JSON. Un *loader* le enseña a procesar otros tipos de archivo antes de incluirlos en el bundle (por ejemplo, convertir un `.css` en algo que JavaScript pueda inyectar en el `<head>`).

```
js

module: {

  rules: [

    { test: /\.css$/, use: ['style-loader', 'css-loader'] }
```

```
  ]
}
```

Ejemplo mental: es como un traductor simultáneo — Webpack "no sabe leer CSS", pero el loader se lo traduce sobre la marcha.

4. Plugins

Mientras los loaders actúan archivo por archivo, los *plugins* actúan sobre el proceso de empaquetado completo: generar automáticamente el `index.html` final, limpiar la carpeta `dist` antes de cada build, extraer el CSS a un archivo aparte, etc.

```
js

plugins: [
  new HtmlWebpackPlugin({ template: './src/index.html' })
]
```

5. Mode (`development` / `production`)

Cambia automáticamente el comportamiento interno: en `development` el bundle es más legible y rápido de generar (para depurar), y en `production` se minifica y optimiza el código para que pese lo menos posible.

6. Code splitting y Hot Module Replacement (HMR)

- *Code splitting*: en vez de generar un único bundle gigante, Webpack puede dividirlo en varios trozos que se cargan sólo cuando hacen falta (por ejemplo, el código de una página que casi nadie visita).
- *HMR*: durante el desarrollo, permite que al guardar un archivo el navegador actualice solo esa pieza de la página, sin recargar todo y sin perder el estado (por ejemplo, el texto que llevabas escrito en un formulario).

Requisitos e instalación

Los requisitos previos son:

- Node.js con versión 18 o superior junto a NPM que se instala junto con Node.
- Un editor de código (p.ej.: Visual Studio Code)
- No hace falta una cuenta en ningún servicio externo, ya que Webpack trabaja en local

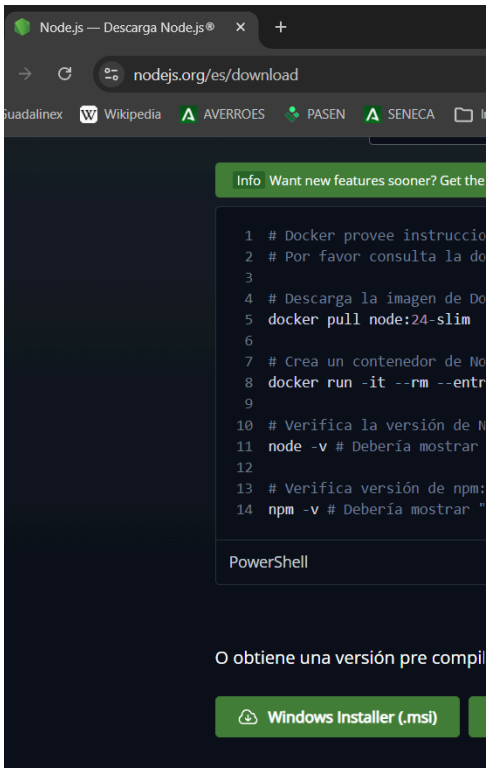
Instalación

Primero para comprobar que tenemos instalado Node y NPM, tenemos que poner en la terminal de Visual Studio Code los siguientes comandos:

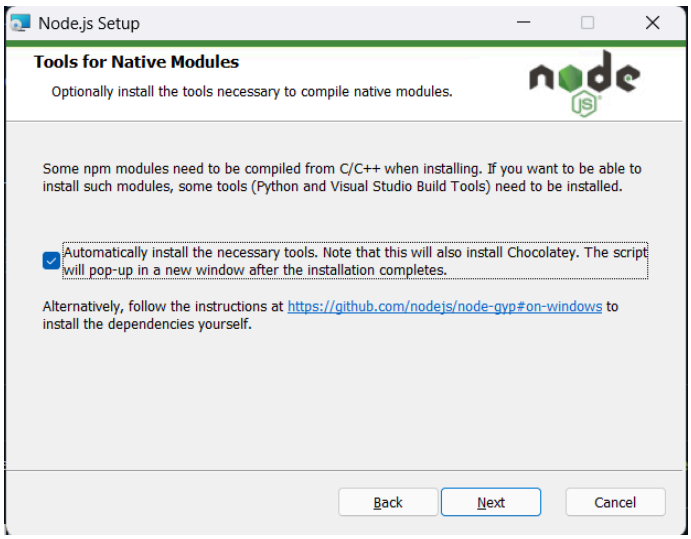
node -v

npm -v

Si no lo tenemos instalado, nos tenemos que ir a la página oficial de Node e instalarlo.



Durante la instalación lo único que tenemos que marcar es que instale todas las herramientas necesarias.



Cuando ya lo tenemos instalado ponemos esos comandos mencionados anteriormente y nos tiene que mostrar las versiones que están instaladas.

```
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> node -v
v24.21.0
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm -v
11.19.0
```

Ahora para inicializar NPM, tenemos que poner el comando “npm init”.

```
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

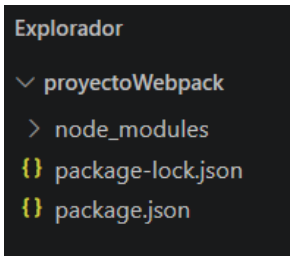
Press ^C at any time to quit.
package name: (proyectowebpack)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack\package.json:

{
  "name": "proyectowebpack",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


Is this OK? (yes)
npm notice
npm notice New major version of npm available! 11.19.0 -> 12.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
npm notice To update run: npm install -g npm@12.2.0
npm notice
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack>
```

Para instalar Webpack, tenemos que poner el siguiente comando “npm install --save-dev webpack webpack-cli ”.

Cuando se instala este paquete, se crean automáticamente dos archivos llamados “package-look.json” y “package.json”.



Y por último tenemos que instalar el plugin que genera el HTML final junto con un servidor de desarrollo. Para ello tenemos que poner el comando “npm install --save-dev html-webpack-plugin webpack-dev-server ”

```
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm install --save-dev webpack webpack-cli

added 100 packages, and audited 101 packages in 11s

17 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm install --save-dev html-webpack-plugin webpack-dev-server

added 223 packages, and audited 324 packages in 13s

105 packages are looking for funding
  run `npm fund` for details

5 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack>
```

Ejemplo básico de uso

En este apartado se va a mostrar un ejemplo básico que se basa en un contador y muestra si es par o impar.

El archivo “index.html” contiene lo siguiente:

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Contador con Webpack</title>

</head>

<body>

  <h1 id="valor">0</h1>

  <p id="estado">Par</p>

  <button id="btn-menos">-1</button>

  <button id="btn-mas">+1</button>



  <!-- No hace falta añadir <script src="bundle.js">: HtmlWebpackPlugin lo
inserta solo -->

</body>

</html>
```

El archivo estilos.css contiene lo siguiente:

```
body {
  font-family: sans-serif;
  text-align: center;
  margin-top: 60px;
  background: #f4f4f6;
  color: #222;
}

#valor {
  font-size: 3rem;
  margin-bottom: 4px;
}

#estado {
  color: #666;
  margin-top: 0;
}

button {
  font-size: 1.2rem;
  margin: 0 8px;
  padding: 6px 18px;
```

```
  border: 1px solid #5fb3a3;
  background: white;
  color: #5fb3a3;
  border-radius: 4px;
  cursor: pointer;
}

button:hover {
  background: #5fb3a3;
  color: white;
}
```

El archivo “contador.js” contiene lo siguiente:

```
export function crearContador(valorInicial = 0) {
  let valor = valorInicial;
  return {
    incrementar: () => ++valor,
    decrementar: () => --valor,
    obtenerValor: () => valor,
    esPar: () => valor % 2 === 0
  };
}
```

El archivo “index.js” contiene lo siguiente:

```
import './estilos.css';                // Webpack incluirá este CSS en el
bundle gracias a los loaders

import { crearContador } from './contador.js';

const contador = crearContador(0);

const valorEl = document.getElementById('valor');

const estadoEl = document.getElementById('estado');

function actualizarVista() {

  valorEl.textContent = contador.obtenerValor();

  estadoEl.textContent = contador.esPar() ? 'Par' : 'Impar';

}

document.getElementById('btn-mas').addEventListener('click', () => {

  contador.incrementar();

  actualizarVista();

});

document.getElementById('btn-menos').addEventListener('click', () => {

  contador.decrementar();

  actualizarVista();

});

actualizarVista();
```

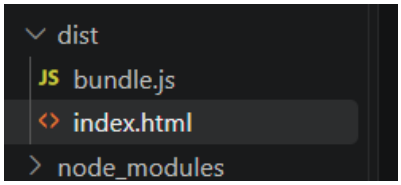
Una vez que tenemos estos archivos listos, tenemos que poner el comando “npm run build ” para compilar solo una vez. Que este genera dist/bundle.js y dist/index.html.

```
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm run build

> webpack-demo@1.0.0 build
> webpack

asset bundle.js 5.26 KiB [emitted] (name: main)
asset index.html 461 bytes [emitted]
runtime modules 614 bytes 3 modules
cacheable modules 1.02 KiB
  ./src/index.js 697 bytes [built] [code generated]
```

Cuando hemos ejecutado ese comando, se ha creado una carpeta llamada “dist”, que dentro de ella se encuentra un archivo llamado “index.html” y otro que se llama “bundle.js”.



Y para levantar el servidor de desarrollo con actualización automática, tenemos que poner el comando “npm start”.

```
PS C:\Users\MSI\Desktop\DAW\optativa\proyectoWebpack> npm start

> webpack-demo@1.0.0 start
> webpack serve

<i> [webpack-dev-server] Project is running at:
<i> [webpack-dev-server] Loopback: http://localhost:8080/, http://[::1]:8080/
<i> [webpack-dev-server] On Your Network (IPv4): http://192.168.100.107:8080/
<i> [webpack-dev-server] Content not from webpack is served from './dist' directory
asset bundle.js 203 KiB [emitted] (name: main)
asset index.html 461 bytes [emitted]
runtime modules 28.7 KiB 13 modules
modules by path ./node_modules/ 124 KiB
  modules by path ./node_modules/webpack-dev-server/client/ 91.6 KiB 8 modules
  modules by path ./node_modules/style-loader/dist/runtime/*.js 5.84 KiB 6 modules
  modules by path ./node_modules/webpack/hot/*.js 5.71 KiB 4 modules
  modules by path ./node_modules/css-loader/dist/runtime/*.js 2.31 KiB 2 modules
  ./node_modules/events/events.js 14.5 KiB [built] [code generated]
  ./node_modules/ansi-html-community/index.js 4.16 KiB [built] [code generated]
modules by path ./src/ 4.11 KiB
  modules by path ./src/*.js 1.02 KiB
    ./src/index.js 697 bytes [built] [code generated]
    ./src/contador.js 351 bytes [built] [code generated]
  modules by path ./src/*.css 3.09 KiB
    ./src/estilos.css 2.23 KiB [built] [code generated]
    ./node_modules/css-loader/dist/cjs.js!./src/estilos.css 877 bytes [built] [code generated]
webpack 5.111.1 compiled successfully in 1057 ms
```

Con eso, ya se abre el navegador con nuestra página lista y funcionando correctamente.

Ventajas e inconvenientes

Algunas de las ventajas son las siguientes:

- Cuenta con un ecosistema enorme de loader y plugins, con prácticamente cualquier tipo de archivo o necesidad que ya tiene una solución hecha.
- Es muy configurable, ya que permite ajustar el comportamiento a proyectos muy distintos, desde una landing page simple hasta una aplicación compleja con múltiples puntos de entrada.
- Es el estándar de facto en el que se apoyan muchas otras herramientas, como Angular CLI o Symphony Encore, por lo que aprenderlo te ayuda a entender "qué hay debajo" de esas herramientas.

Algunas de las desventajas son:

- Tiene una curva de aprendizaje notable, con la configuración “webpack.config.js” puede resultar intimidante al principio, ya que cuenta con muchos conceptos que aprender a la vez.
- Algunas herramientas más modernas como Vite suelen ser más rápidas en desarrollo y requieren menos configuración para casos sencillos.
- Y los mensajes de error pueden ser difíciles de interpretar para alguien que está empezando, ya que combinan varios loaders.

Conclusión y valoración personal Webpack sigue siendo una herramienta muy sólida y ampliamente documentada, especialmente útil cuando se necesita un control fino sobre cómo se empaqueta un proyecto. Para un proyecto nuevo y sencillo, hoy en día probablemente optaría por una alternativa más ligera como Vite; pero en un proyecto real con requisitos de configuración específicos, o que ya forme parte de un stack que lo usa internamente (como Angular), seguiría usando Webpack sin problema.

Recursos y bibliografía

- Documentación oficial de Webpack: https://webpack.js.org/
- Guía de conceptos (Concepts): https://webpack.js.org/concepts/
- Guía paso a paso "Getting Started": https://webpack.js.org/guides/getting-started/
- Repositorio oficial en GitHub: https://github.com/webpack/webpack
- Documentación de HtmlWebpackPlugin: https://github.com/jantimon/html-webpack-plugin
- Documentación de webpack-dev-server: https://webpack.js.org/configuration/dev-server/

Página de Git con el proyecto: https://github.com/atorcas14/manual-webpack-torres